Individual Contribution Report

MiniOS Omega (1063) - Operating Systems Final Project

Student Name:	Zaiden Rianne Bibanco
Group:	MiniOS Omega (1063)
Assigned Role:	System Developer (Error Handling, Testing, Documentation Assistance)

What did I contribute to the project?

As the System Developer, I was responsible for making MiniOS reliable and well documented. I worked on the error handling of the system, tested every command to confirm it behaved correctly, and helped prepare the project documentation, including the User Manual.

What features did I personally develop?

- **Error handling:** the system responds to an unknown command with "Unknown command. Type 'help' to see the available commands." instead of crashing.
- **Input validation messages** for commands that need arguments, such as usage hints (`Usage: mkdir <folder>`), and errors such as "File does not exist", "Process does not exist", "PID must be a number", "Not enough free memory" and "Cannot free more memory than allocated".
- **Safety net in the command loop:** unexpected errors are caught and shown as an `Error:` message, and the prompt returns so the user can keep working.
- **Testing:** running each of the commands with valid and invalid input and recording the results.
- **Documentation:** the User Manual (`User_Manual.pdf`), covering installation, login, every command with purpose, syntax and example, common errors, and how to exit.

What problems did I encounter?

- Invalid input such as missing arguments, non-numeric PIDs or negative memory amounts could cause confusing behavior or crashes.
- Keeping error messages consistent across the files written by different members (main, filesystem, processes, memory).
- Making sure the documentation matched what the program really does, including small details such as that password typing is hidden and names cannot contain spaces.

How did I solve those problems?

- Checked every command for missing, wrong-type and out-of-range input, and used clear messages in one common style (`Error: ...` and `Usage: ...`).
- Ran the program repeatedly with deliberately wrong input and reported issues to the member who owned that module.
- Read through the source code while writing the manual so every command, message and example reflects the real behavior.

What did I learn from this project?

I learned how the parts of an operating system work together, how a command interpreter reads and dispatches commands, and why simulating processes, memory and files still needs careful rules. I also learned that handling errors well matters as much as the main feature, and that testing and clear documentation make a system usable by other people.

Evidence of my contribution:

- Error-handling code in the MiniOS source files (`main.py`, `filesystem.py`, `processes.py`, `memory.py`).
- `docs/User_Manual.pdf` in the group's GitHub repository.
- Screenshots of error messages and test runs in the `screenshots/` folder.
- My commits in the group's GitHub repository.